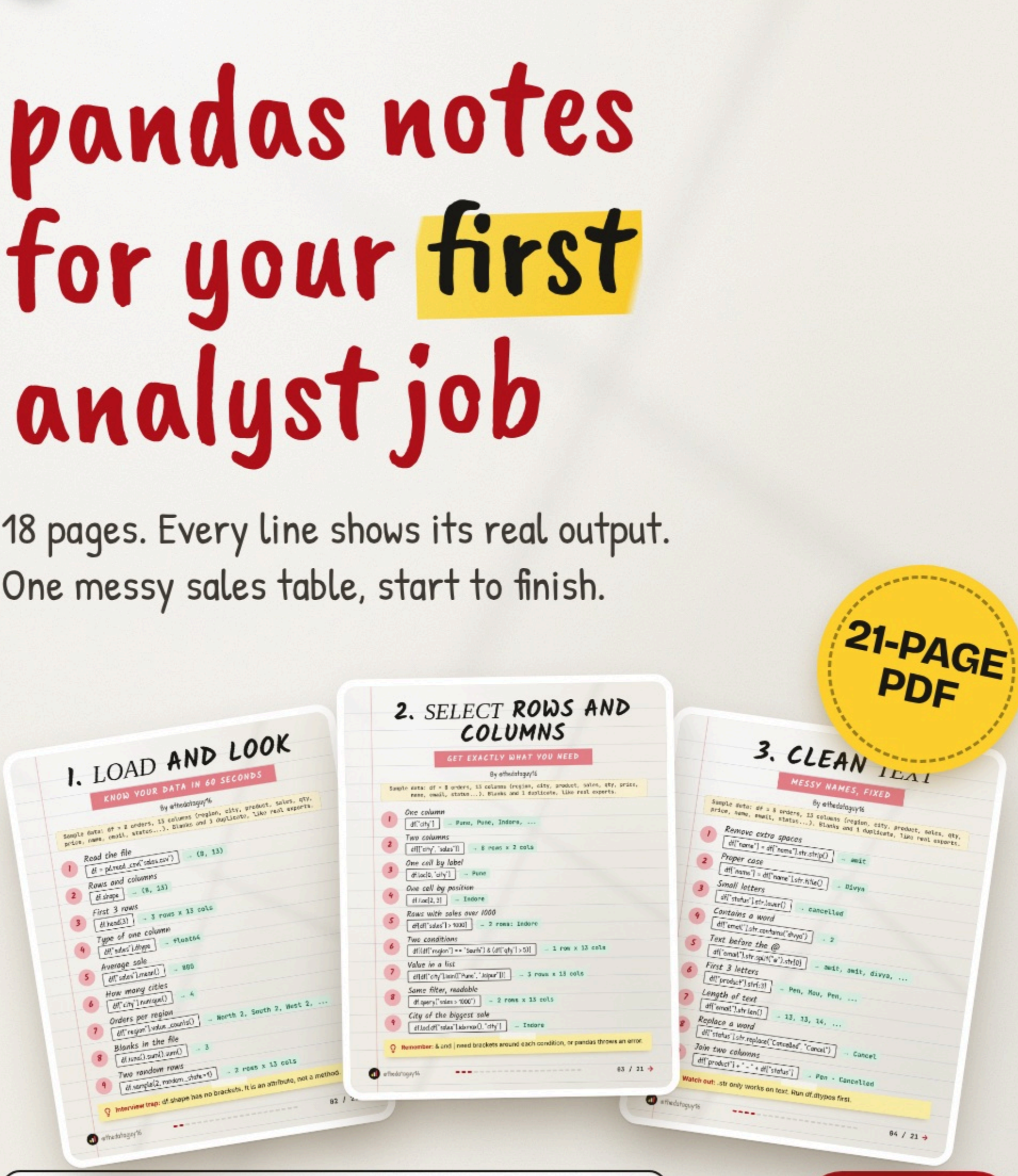
thedataguy16

SAVE FOR LATER

pandas notes for your first analyst job

18 pages. Every line shows its real output.
One messy sales table, start to finish.

21-PAGE PDF



Comment PANDASNOTES for the PDF

Swipe →

1. LOAD AND LOOK

KNOW YOUR DATA IN 60 SECONDS

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Read the file*

```
df = pd.read_csv("sales.csv")
```
→ (8, 13)

2 *Rows and columns*

```
df.shape
```
→ (8, 13)

3 *First 3 rows*

```
df.head(3)
```
→ 3 rows x 13 cols

4 *Type of one column*

```
df["sales"].dtype
```
→ float64

5 *Average sale*

```
df["sales"].mean()
```
→ 800

6 *How many cities*

```
df["city"].nunique()
```
→ 4

7 *Orders per region*

```
df["region"].value_counts()
```
→ North 2, South 2, West 2, ...

8 *Blanks in the file*

```
df.isna().sum().sum()
```
→ 3

9 *Two random rows*

```
df.sample(2, random_state=1)
```
→ 2 rows x 13 cols

Interview trap: df.shape has no brackets. It is an attribute, not a method.

2. SELECT ROWS AND COLUMNS

GET EXACTLY WHAT YOU NEED

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *One column*

```
df["city"]
```
→ Pune, Pune, Indore, ...

2 *Two columns*

```
df[["city", "sales"]]
```
→ 8 rows x 2 cols

3 *One cell by label*

```
df.loc[0, "city"]
```
→ Pune

4 *One cell by position*

```
df.iloc[2, 3]
```
→ Indore

5 *Rows with sales over 1000*

```
df[df["sales"] > 1000]
```
→ 2 rows: Indore

6 *Two conditions*

```
df[(df["region"] == "South") & (df["qty"] > 5)]
```
→ 1 row x 13 cols

7 *Value in a list*

```
df[df["city"].isin(["Pune", "Jaipur"])]
```
→ 3 rows x 13 cols

8 *Same filter, readable*

```
df.query("sales > 1000")
```
→ 2 rows x 13 cols

9 *City of the biggest sale*

```
df.loc[df["sales"].idxmax(), "city"]
```
→ Indore

Remember: & and | need brackets around each condition, or pandas throws an error.

3. CLEAN TEXT

MESSY NAMES, FIXED

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Remove extra spaces*

```
df["name"] = df["name"].str.strip()
```
→ amit

2 *Proper case*

```
df["name"] = df["name"].str.title()
```
→ Divya

3 *Small letters*

```
df["status"].str.lower()
```
→ cancelled

4 *Contains a word*

```
df["email"].str.contains("divya")
```
→ 2

5 *Text before the @*

```
df["email"].str.split("@").str[0]
```
→ amit, amit, divya, ...

6 *First 3 letters*

```
df["product"].str[:3]
```
→ Pen, Mou, Pen, ...

7 *Length of text*

```
df["email"].str.len()
```
→ 13, 13, 14, ...

8 *Replace a word*

```
df["status"].str.replace("Cancelled", "Cancel")
```
→ Cancel

9 *Join two columns*

```
df["product"] + " - " + df["status"]
```
→ Pen - Cancelled

Watch out: .str only works on text. Run df.dtypes first.

4. MISSING VALUES

BLANKS BREAK EVERY AVERAGE

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Blanks per column*

```
df.isna().sum()
```
→ region 1, city 1, sales 1

2 *Rows with any blank*

```
df[df.isna().any(axis=1)]
```
→ 2 rows x 13 cols

3 *Drop rows missing sales*

```
df.dropna(subset=["sales"])
```
→ 7 rows x 13 cols

4 *Fill blank sales with 0*

```
df["sales"].fillna(0)
```
→ 0

5 *Fill with the average*

```
df["sales"].fillna(df["sales"].mean())
```
→ 800

6 *Fill blank city*

```
df["city"].fillna("Unknown")
```
→ Unknown

7 *Copy the value above*

```
df["region"].ffill()
```
→ West

8 *Count filled cells*

```
df["sales"].count()
```
→ 7

9 *Share of blanks*

```
df["sales"].isna().mean()
```
→ 0.125

Interview trap: count() skips blanks, len(df) does not. 7 vs 8 here.

5. DUPLICATES

DOUBLE COUNTING KILLS REPORTS

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *How many duplicate rows*

```
df.duplicated().sum()
```
→ 1

2 *Remove them*

```
df = df.drop_duplicates()
```
→ 7

3 *Duplicates in one column*

```
df.duplicated(subset=["email"]).sum()
```
→ 2

4 *Keep the latest per email*

```
df.drop_duplicates(subset=["email"], keep="last")
```
→ 5 rows x 13 cols

5 *See every repeat*

```
df[df.duplicated(subset=["email"], keep=False)]
```
→ 4 rows x 13 cols

6 *Unique cities*

```
df["city"].dropna().unique()
```
→ 4

7 *Renumber the rows*

```
df = df.reset_index(drop=True)
```
→ 0 to 6

Remember: drop_duplicates() returns a copy. Save it back with df = ...

6. DATES AND TYPES

TURN TEXT INTO REAL DATES

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Text to date*

```
df["date"] = pd.to_datetime(df["date"])
```
→ datetime64

2 *Month number*

```
df["date"].dt.month
```
→ 1, 1, 2, ...

3 *Day name*

```
df["date"].dt.day_name()
```
→ Wednesday, Monday, Saturday...

Month label

4

```
df["date"].dt.strftime("%b %Y")
```
→ Jan 2024, Jan 2024, Feb 202...

5 *Orders in March*

```
df[df["date"].dt.month == 3]
```
→ 4 rows x 13 cols

6 *Days since first order*

```
(df["date"] - df["date"].min()).dt.days
```
→ 0, 12, 24, ...

7 *Number stored as text*

```
pd.to_numeric(pd.Series(["10", "x"]), errors="coerce")
```
→ 10, NaN

8 *Whole numbers*

```
df["qty"].astype(int)
```
→ 5, 2, 20, ...

Interview trap: errors="coerce" turns bad values into NaN instead of crashing.

7. NEW COLUMNS

REVENUE, FLAGS, BANDS

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Revenue*

```
df["revenue"] = df["qty"] * df["price"]
```

→ 500, 900, 1200, ...

2 *Yes / no flag*

```
df["big"] = np.where(df["revenue"] > 1000, "Yes", "No")
```

→ No, No, Yes, ...

3 *Price bands*

```
pd.cut(df["price"], bins=[0, 100, 500, 1000], labels=["Low", "Mid", "High"])
```

→ Low, Mid, Low, ...

4 *Map codes to names*

```
df["status"].map({"Delivered": "D", "Cancelled": "C", "Returned": "R"})
```

→ D, D, C, ...

5 *Round to 1 decimal*

```
(df["sales"] / 3).round(1)
```

→ 166.7, 300, 400, ...

6 *Cap extreme values*

```
df["qty"].clip(upper=20)
```

→ 5, 2, 20, ...

7 *Running total*

```
df["revenue"].cumsum()
```

→ 500, 1400, 2600, ...

8 *Share of total*

```
(df["revenue"] / df["revenue"].sum() * 100).round(1)
```

→ 6.2, 11.2, 15, ...

Faster: np.where beats apply(lambda) on big files. Same answer.

8. GROUP AND SUMMARISE

SUMIF, COUNTIF, AVERAGEIF IN ONE LINE

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Sales by region*

```
df.groupby("region")["sales"].sum()
```

→ East 300, North 1400, South...

2 *Average by region*

```
df.groupby("region")["sales"].mean()
```

→ East 300, North 700, South ...

3 *Orders per city*

```
df.groupby("city").size()
```

→ Indore 2, Jaipur 1, Lucknow...

4 *Many numbers at once*

```
df.groupby("region")["sales"].agg(["sum", "mean", "count"])
```

→ 4 rows x 3 cols

5 *Named results*

```
df.groupby("region").agg(total=("sales", "sum"), orders=("order_id", "count"))
```

→ 4 rows x 2 cols

6 *Biggest region first*

```
df.groupby("region")["sales"].sum().sort_values(ascending=False)
```

→ South 3000, North 1400, Wes...

7 *Unique customers per region*

```
df.groupby("region")["customer_id"].nunique()
```

→ East 1, North 1, South 1, ...

8 *Each row's share of its region*

```
df["sales"] / df.groupby("region")["sales"].transform("sum")
```

→ 0.3571, 0.6429, 0.4, ...

Interview trap: groupby drops blank keys. The order with no region vanished.

9. PIVOT AND RESHAPE

EXCEL PIVOTS, IN CODE

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

Pivot: region x month

1

```
df.pivot_table(index="region", columns="month", values="sales",
aggfunc="sum", fill_value=0)
```

→ 4 rows x 3 cols

With totals

2

```
df.pivot_table(index="region", values="sales", aggfunc="sum", margins=True)
```

→ 5 rows x 1 cols

Count table

3

```
pd.crosstab(df["region"], df["status"])
```

→ 4 rows x 3 cols

Wide to long

4

```
df[["order_id", "sales", "qty"]].melt(id_vars="order_id")
```

→ 16 rows x 3 cols

Grand total

5

```
df.pivot_table(index="region", values="sales", aggfunc="sum",
margins=True).loc["All", "sales"]
```

→ 5600

Remember: fill_value=0 shows 0 instead of NaN in empty pivot cells.

10. JOIN TABLES

VLOOKUP, WITHOUT THE PAIN

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

Bring in the segment

1

```
df.merge(customers, on="customer_id", how="left")
```

→ 8 rows x 14 cols

Only matching rows

2

```
df.merge(customers, on="customer_id", how="inner")
```

→ 7 rows x 14 cols

Who has no segment?

3

```
df.merge(customers, on="customer_id", how="left")["segment"].isna().sum()
```

→ 1

See where rows came from

4

```
df.merge(customers, on="customer_id", how="outer", indicator=True)
["_merge"].value_counts()
```

→ both 7, left_only 1, right_...

Stack two files

5

```
pd.concat([df, df], ignore_index=True)
```

→ 16 rows x 13 cols

Different column names

6

```
df.merge(customers.rename(columns={"customer_id": "cid"}),
left_on="customer_id", right_on="cid")
```

→ 7 rows x 15 cols

Interview trap: a left join can ADD rows if the right table has repeats. Check len() after.

11. SORT AND RANK

TOP SELLERS, BOTTOM SELLERS

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Highest sales first*

```
df.sort_values("sales", ascending=False)
```

→ 1800

Two columns

2

```
df.sort_values(["region", "sales"], ascending=[True, False])
```

→ 8 rows x 13 cols

3 *Top 3 orders*

```
df.nlargest(3, "sales")
```

→ [1800, 1200, 900]

4 *Bottom 2*

```
df.nsmallest(2, "sales")
```

→ [300, 450]

5 *Rank, biggest = 1*

```
df["sales"].rank(ascending=False)
```

→ [7, 5.5, 5.5]

6 *Rank with no gaps*

```
df["sales"].rank(method="dense", ascending=False)
```

→ [6, 5, 5]

Best order per region

7

```
df.sort_values("sales").groupby("region").tail(1)
```

→ 4 rows x 13 cols

Remember: method="dense" gives 1, 2, 2, 3. The default gives 1, 2.5, 2.5, 4.

12. TIME SERIES

MONTH ON MONTH, THE ANALYST WAY

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

Monthly total

1

```
df.resample("MS", on="date")["sales"].sum()
```

→ Jan 1400, Feb 3000, Mar 1200

Change vs last month

2

```
df.resample("MS", on="date")["sales"].sum().diff()
```

→ Jan NaN, Feb 1600, Mar -1800

Growth %

3

```
(df.resample("MS", on="date")["sales"].sum().pct_change() * 100).round(1)
```

→ Jan NaN, Feb 114.3, Mar -60

Previous value

4

```
df["sales"].shift(1)
```

→ NaN, 500, 900, ...

2-order moving average

5

```
df["sales"].rolling(2).mean()
```

→ NaN, 700, 1050, ...

Running total by region

6

```
df.groupby("region")["sales"].cumsum()
```

→ 500, 1400, 1200, ...

Interview trap: resample needs a datetime column. Convert first.

13. EXPORT YOUR WORK

HAND IT TO YOUR MANAGER

By @thedataguy16

Sample data: df = 8 orders, 13 columns (region, city, product, sales, qty, price, name, email, status...). Blanks and 1 duplicate, like real exports.

1 *Save as CSV*

```
df.to_csv("out/report.csv", index=False)
```

→ True

Save as Excel

2

```
df.to_excel("out/report.xlsx", sheet_name="Sales", index=False)
```

→ True

Two sheets, one file

3

```
with pd.ExcelWriter("out/two.xlsx") as w: df.to_excel(w,
sheet_name="All")
```

→ 1 file, sheet All

Round before you send

4

```
df.round(2).to_csv("out/clean.csv", index=False)
```

→ True

Check what you saved

5

```
pd.read_csv("out/report.csv").shape
```

→ (8, 13)

Remember: index=False, or your file gets an extra unnamed column.

14. SQL to *pandas*

by @thedataguy16

#	SQL	pandas
1	SELECT city, sales	`df[["city", "sales"]]`
2	WHERE sales > 1000	`df[df["sales"] > 1000]`
3	ORDER BY sales DESC	`df.sort_values("sales", ascending=False)`
4	LIMIT 5	`df.head(5)`
5	SELECT DISTINCT city	`df["city"].drop_duplicates()`
6	COUNT(*)	`len(df)`
7	GROUP BY region, SUM(sales)	`df.groupby("region")["sales"].sum()`
8	HAVING SUM(sales) > 1000	`df.groupby("region")["sales"].sum().loc[lambda s: s > 1000]`
9	LEFT JOIN customers USING id	`df.merge(df2, on="customer_id", how="left")`
10	UNION ALL	`pd.concat([df, df])`
11	WHERE city IN (...)	`df[df["city"].isin(["Pune", "Indore"])]`
12	WHERE name LIKE '%a%'	`df[df["name"].str.contains("a")]`
13	WHERE city IS NULL	`df[df["city"].isna()]`
14	CASE WHEN ... THEN	`np.where(df["sales"] > 1000, "High", "Low")`
15	AS (rename)	`df.rename(columns={"sales": "amount"})`
16	ROW_NUMBER() OVER	`df.groupby("region").cumcount() + 1`

15. Excel to *pandas*

by @thedataguy16

#	Excel	pandas
1	VLOOKUP	`df.merge(df2, on="customer_id", how="left")`
2	SUMIF	`df.groupby("region")["sales"].sum()`
3	COUNTIF	`df["region"].value_counts()`
4	AVERAGEIF	`df.groupby("region")["sales"].mean()`
5	IF	`np.where(df["sales"] > 1000, "Yes", "No")`
6	Pivot table	`df.pivot_table(index="region", values="sales", aggfunc="sum")`
7	Remove duplicates	`df.drop_duplicates()`
8	Filter	`df[df["city"] == "Pune"]`
9	Sort A to Z	`df.sort_values("name")`
10	TRIM	`df["name"].str.strip()`
11	PROPER	`df["name"].str.title()`
12	LEFT(A2, 3)	`df["product"].str[:3]`
13	Text to columns	`df["email"].str.split("@", expand=True)`
14	IFERROR(x, 0)	`df["sales"].fillna(0)`
15	Running total	`df["sales"].cumsum()`
16	RANK	`df["sales"].rank(ascending=False)`

16. Errors and fixes

by @thedataguy16

#	Error	Usually means / fix
1	KeyError: 'Sales'	Column names are case sensitive. Check df.columns.
2	SettingWithCopyWarning	Use df.loc[rows, col] = value, not chained [][].
3	ValueError on merge	Key columns have different types. astype(str) both.
4	Can only use .str accessor	Column is not text. Check df.dtypes.
5	TypeError: unsupported operand	Numbers stored as text. Use pd.to_numeric.
6	Length mismatch	New column has a different number of rows.
7	Unnamed: 0 column	File saved with index. Use index=False.
8	Wrong totals after join	Right table had duplicate keys. Check len().
9	NaN in results	Blanks upstream. Run df.isna().sum().
10	Dates sort wrong	Still text. Use pd.to_datetime first.
11	MemoryError on big files	Read only needed columns: usecols=[...].
12	Empty DataFrame after filter	Extra spaces in text. str.strip() first.

17. Interview questions, short answers

by @thedataguy16

#	Question	Answer
1	loc vs iloc?	loc uses labels, iloc uses positions.
2	merge vs concat?	merge joins on keys, concat stacks tables.
3	How do you find duplicates?	df.duplicated().sum(), then drop_duplicates().
4	apply vs vectorised code?	Vectorised (np.where, +, *) is much faster.
5	What does groupby return?	Groups you summarise with sum, mean, agg.
6	Handle missing values?	Find with isna(), then drop or fill with a reason.
7	count() vs size()?	count skips blanks, size counts every row.
8	Why reset_index()?	Turns the group labels back into columns.
9	How to read a big CSV?	usecols, dtype, or chunksize.
10	pivot vs pivot_table?	pivot_table aggregates repeats, pivot cannot.
11	transform vs agg?	transform keeps every row, agg shrinks them.
12	Series vs DataFrame?	One column vs a full table.
13	Why inplace=True is avoided?	Reassigning (df = ...) is clearer and safer.
14	How to change a type?	astype(), pd.to_numeric(), pd.to_datetime().

18. Practise these 12 tasks

by @thedataguy16

#	Task	You will use
1	Load a sales CSV and check blanks	`read_csv, isna`
2	Clean messy customer names	`str.strip, str.title`
3	Remove duplicate orders	`drop_duplicates`
4	Revenue = qty x price	`new column`
5	Sales by region and month	`groupby, pivot_table`
6	Add customer segment from another file	`merge`
7	Top 3 products per region	`sort_values, groupby head`
8	Month-on-month growth	`resample, pct_change`
9	Flag orders above 1000	`np.where`
10	Fix dates stored as text	`pd.to_datetime`
11	Share of total per region	`transform`
12	Export a clean Excel report	`to_excel`

☆ THE 90-DAY SYSTEM · 5.0 BEST SELLER

Become a data analyst in 90 days.

Open it every morning. It tells you what to learn, build and ship that day. 1 hour a day, no coding background needed.

DAY 1-21	DAY 22-45	DAY 46-65	DAY 66-90
Excel + Stats	SQL	Power BI	Python + Jobs

27 portfolio projects with real data

100+ interview Q&As with model answers

365 days of 1:1 mentor access

10+ job applications sent by Day 90

0 to job-ready, one day at a time

Built by Abhishek · 7 yrs in data · 31,000+ helped

Link in bio →

One page a day.

Comment PANDASNOTES and I'll DM you the full 21-page PDF.



Follow @thedataguy16

Send to a friend

@thedataguy16

Save this